

Proposal Report

프로젝트 작품 제목

<차량 안전 제어 시스템>

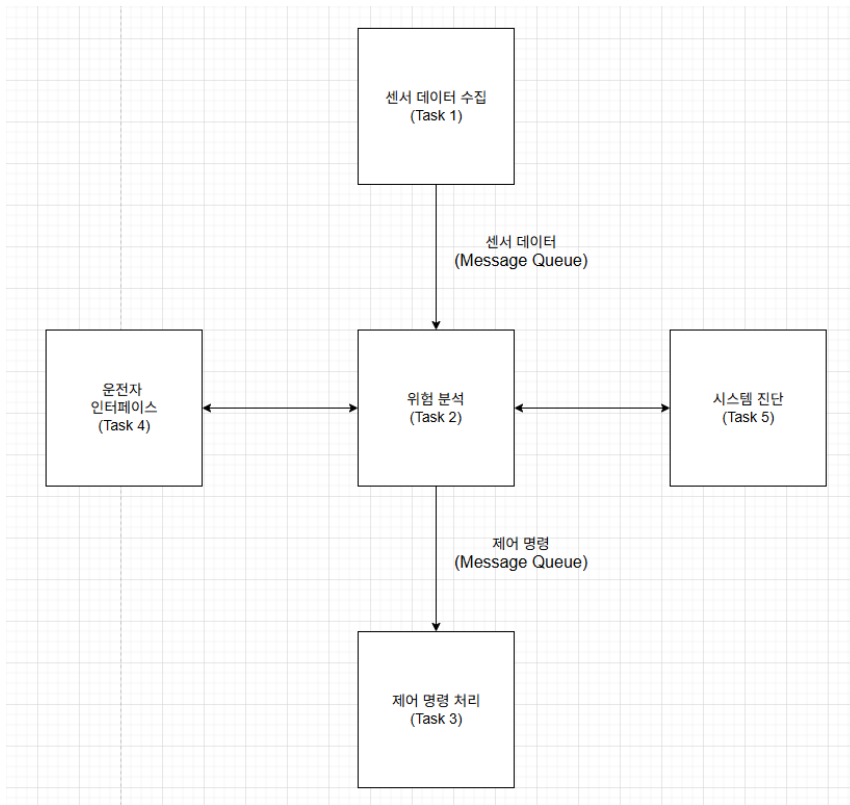
과 목	임베디드시스템S/W설계
담당교수	김태석 교수님
학 과	컴퓨터공학과
학 번	2019202084
성 명	배정우
날 짜	2025. 05. 13 (화요일)



1. 요약

차량 안전 제어 시스템은 다양한 센서로부터 데이터를 수집하여 잠재적인 위험 상황을 감지하고, 이에 대응하여 차량 제어 시스템에 명령을 전달하는 실시간 임베디드 시스템이다. 이 시스템은 충돌 위험, 차선 이탈, 과속 등의 위험 상황을 탐지하고 경고를 생성하거나 자동 개입을 통해 사고를 예방한다.

<프로젝트 전반을 설명하는 그림>



2. 본문

▶ RTOS 필요성

차량 안전 제어 시스템에서 RTOS는 다음과 같은 이유로 필요하다.

- 1) 실시간 응답성 → 위험 상황은 즉각적인 대응이 필요하다. RTOS는 정해진 시간 내에 task 실행을 보장하여 치명적인 지연을 방지할 수 있다.
- 2) 우선순위 기반 스케줄링 → 충돌 감지와 같은 고위험 작업에 더 높은 우선순위를 부여하여 주요한 안전 기능이 다른 작업에 의해 지연되지 않도록 한다.
- 3) 리소스 동기화 → 여러 센서 데이터와 제어 명령이 동시에 처리될 때 자원 접근 충돌을 방지한다.
- 4) 인터럽트 처리 → 센서로부터 긴급 신호를 즉시 처리할 수 있는 효율적인 인터럽트 처리 메커니즘을 제공한다.
- 5) 신뢰성 → 차량 안전 시스템은 높은 신뢰성을 요구하며, RTOS는 안정적인 작업 관리를 통해 이를 지원한다.

▶ 개발할 kernel function

동적 우선순위 조정 함수(OS_DynamicPrioAdjust) → 위험 수준에 따라 task의 우선순위를 동적으로 조정하는 기능을 제공하려고 한다. 시스템이 높은 위험 상황을 감지하면, 관련 task의 우선순위를 일시적으로 높여 더 빠르게 처리될 수 있도록 한다.

<개발 필요성>

기존 uC/OS-II는 task의 우선순위를 런타임에 변경할 수 있지만, 시간 기반의 자동 복원 기능은 없는 걸로 알고 있다. 위험 상황에서 일시적으로 우선순위를 높이고, 위험이 지나면 자동으로 원래 우선순위로 복원하는 기능이 있다면 안전 제어 시스템에 유용할 것이다.

<예제>

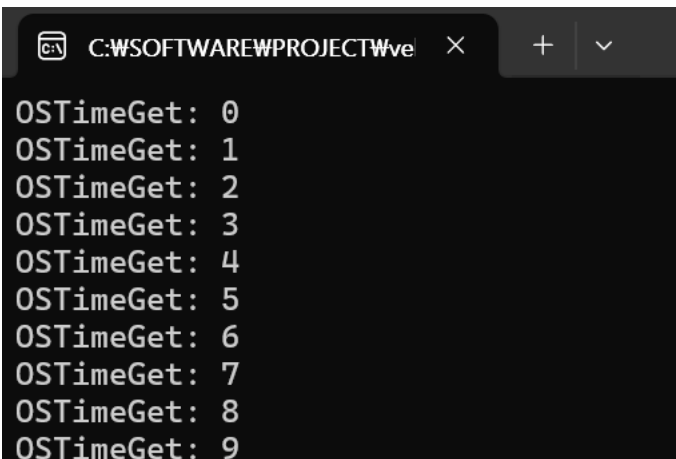
```
#if OS_TIME_GET_SET_EN > 0
INT32U OSTimeGet (void)
{
    #if OS_CRITICAL_METHOD == 3
        OS_CPU_SR  cpu_sr;
    #endif
    INT32U      ticks;

    OS_ENTER_CRITICAL();
    ticks = OSTime;

    printf("OSTimeGet: %d\n", ticks);

    OS_EXIT_CRITICAL();
    return (ticks);
}
#endif
```

```
1  #include "INCLUDES.H"
2  #define TASK_STK_SIZE 512
3
4  // Task Stack Memory 지정
5  OS_STK TaskStk1[TASK_STK_SIZE]; // 센서 수집
6
7  // 센서 데이터 수집 태스크
8  void SensorTask(void *pdata) {
9      while (1) {
10         OSTimeGet(); // 커널 함수 호출
11         OSTimeDlyHMSM(0, 0, 1, 0); // 1초 delay
12     }
13 }
14
15 int main(void) {
16     OSInit(); // uC/OS-II 커널을 초기화
17
18     // 태스크 생성 및 우선순위 부여
19     OSTaskCreate(SensorTask, (void *)0, &TaskStk1[TASK_STK_SIZE-1], 2);
20     OSStart(); // 멀티 태스킹 시작
21
22     return 0;
23 }
```



```
C:\SOFTWARE\PROJECT\Wve  X  +  v
OSTimeGet: 0
OSTimeGet: 1
OSTimeGet: 2
OSTimeGet: 3
OSTimeGet: 4
OSTimeGet: 5
OSTimeGet: 6
OSTimeGet: 7
OSTimeGet: 8
OSTimeGet: 9
```

위의 사진은 uC/OS-II에서 커널 함수 내부에 디버깅 문자열을 삽입하고, 이를 태스크에서 호출했다. 좌측 상단의 사진은 OS_TIME.C 내부에 정의된 커널 함수인 OSTimeGet()에 printf("OSTimeGet: %d\n", ticks); 문장을 삽입하여 현재 커널 타이머 값을 콘솔에 출력하도록 수정하였다. 이는 태스크에서 커널 함수가 호출될 때마다 로그가 찍힌다. 우측 상단의 사진은 태스크인 SensorTask()이다. 이 태스크는 주기적으로 OSTimeGet()을 호출하며, 호출 시마다 커널 함수 내부에서 printf()를 통해 현재 시간(tick)을

출력하게 된다. OSTimeDlyHMSM(0, 0, 1, 0)을 통해 1초 간격으로 커널 함수를 반복 호출한다. 마지막으로 하단의 콘솔 출력 사진은 nmake -f makefile 명령어를 사용해 실행한 결과를 보여준다. 실행 결과 OSTimeGet() 함수가 호출될 때마다 커널 함수 내부의 printf()가 작동하여 시스템 시간(tick 값)이 출력된다. 출력 결과는 1초마다 증가하는 시스템 tick 값을 잘 나타내는 것을 확인할 수 있다.

▶ 각 task의 정의

1) 센서 데이터 수집 태스크

- 역할 → 각종 센서로부터 데이터를 수집하고 위험 분석 태스크로 전달
- 우선순위 → 높음 → 센서 데이터는 시스템 동작이 되므로 높은 우선순위 필요

2) 위험 분석 태스크

- 역할 → 센서 데이터를 분석하여 충돌 가능성, 차선 이탈 등의 위험 상황을 판단
- 우선순위 → 가장 높음 → 위험 상황 판단은 최우선으로 처리되어야 함

3) 제어 명령 처리 태스크

- 역할 → 위험 분석 결과에 따라 브레이크, 스티어링 휠 조작 등 제어 명령 생성 및 전송
- 우선순위 → 높음 → 위험 상황 대응을 위해 높은 우선순위 필요

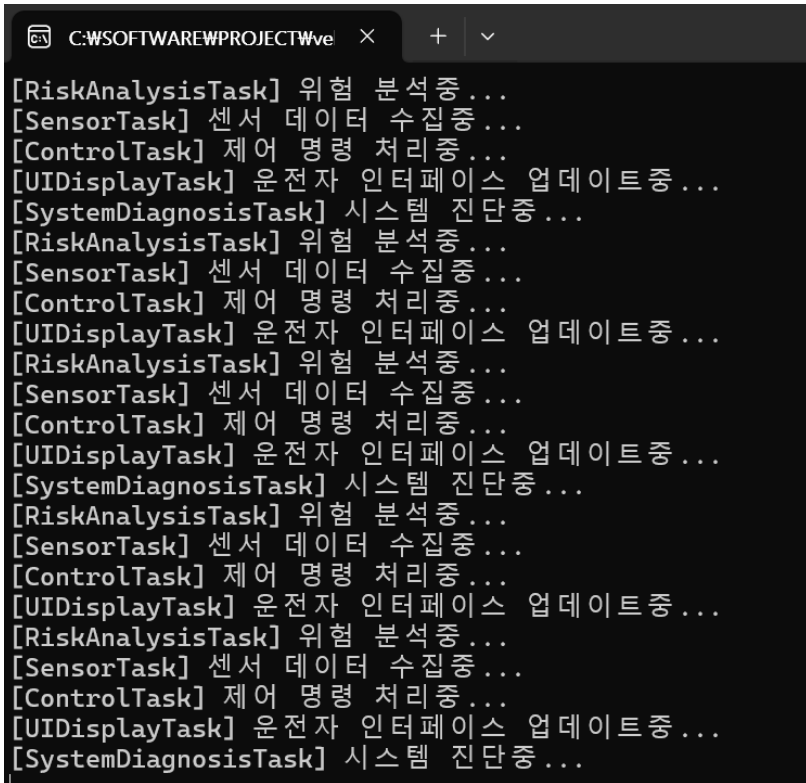
4) 운전자 인터페이스 태스크

- 역할 → 경고음, 경고등, 디스플레이 메시지 등을 통해 운전자에게 정보 전달
- 우선순위 → 중간 → 운전자에게 알림은 중요하나 제어보다는 우선순위가 낮음

5) 시스템 진단 태스크

- 역할 → 시스템 상태 모니터링, 오류 감지, 로깅 수행
- 우선순위 → 낮음 → 백그라운드에서 실행되는 지원 기능

▶ task 동작 결과



```
C:\SOFTWAREPROJECT\Wve x + v
[RiskAnalysisTask] 위험 분석중...
[SensorTask] 센서 데이터 수집중...
[ControlTask] 제어 명령 처리중...
[UIDisplayTask] 운전자 인터페이스 업데이트중...
[SystemDiagnosisTask] 시스템 진단중...
[RiskAnalysisTask] 위험 분석중...
[SensorTask] 센서 데이터 수집중...
[ControlTask] 제어 명령 처리중...
[UIDisplayTask] 운전자 인터페이스 업데이트중...
[RiskAnalysisTask] 위험 분석중...
[SensorTask] 센서 데이터 수집중...
[ControlTask] 제어 명령 처리중...
[UIDisplayTask] 운전자 인터페이스 업데이트중...
[SystemDiagnosisTask] 시스템 진단중...
[RiskAnalysisTask] 위험 분석중...
[SensorTask] 센서 데이터 수집중...
[ControlTask] 제어 명령 처리중...
[UIDisplayTask] 운전자 인터페이스 업데이트중...
[RiskAnalysisTask] 위험 분석중...
[SensorTask] 센서 데이터 수집중...
[ControlTask] 제어 명령 처리중...
[UIDisplayTask] 운전자 인터페이스 업데이트중...
[SystemDiagnosisTask] 시스템 진단중...
```

위는 5개의 task를 생성하고 동시에 실행하는 멀티 태스킹을 하고 있다. RiskAnalysisTask는 우선순위 값을 1로 주어 5개의 task 중 우선순위가 제일 높고, SensorTask는 우선순위 값을 2로 주고, ControlTask는 우선순위 값을 3으로 주고, UIDisplayTask는 우선순위 값을 4로 주고, SystemDiagnosisTask는 우선순위 값을 5로 주었다. RiskAnalysisTask는 가장 높은 우선순위로 설정되어 있으므로 다른 태스크들보다 먼저 실행이 되었다. 나머지 태스크들도 우선순위에 따라 실행되었다. 태스크들 모두 OSTimeDlyHMSM() 함수를 사용하여 지연을 발생시켰는데 SystemDiagnosisTask만 2초로 주었고 나머지 태스크들은 1초로 주었다. 그래서 위의 사진을 보면 SystemDiagnosisTask는 2초마다 실행되므로 출력빈도가 다른 태스크들보다 절반인 것을 확인할 수 있다.